RoastMyIdea Design Document

Authors: Achyut Katiyar, Soni Rusagara
Class: CS5610 Web Development
Date: March 19th 2026

Project Description

RoastMyIdea is a platform where users pitch ideas and the community decides if they are worth building. Anyone can post an idea whether it is a startup concept or a silly invention. Other users can roast the idea by writing a critique or defend it by replying to a roast with a counter argument. Users can also invest a virtual currency called RoastCoin into ideas they believe in. After 7 days the idea gets a verdict. If the defenses got more upvotes it survives and backers earn a 1.5x return. If the roasts won then the idea is torched and backers lose their investment. If it is too close to call the idea goes lukewarm and everyone gets refunded. It is a fun and gamified way to pressure test ideas through real community debate. The app uses Node.js, Express, MongoDB, Passport.js, and React with hooks.

User Personas

Nate is a college student who always has random app ideas but never knows if they are actually good. He wants a place to pitch them and get real feedback before spending time building something nobody wants.

Priya works in product management and loves finding flaws in ideas. She enjoys breaking down why something will not work and wants a space where that kind of honest critique is encouraged not frowned upon.

Marco is the kind of person who gets excited about other peoples ideas. When he sees something creative he wants to back it and argue for it. He likes the idea of putting his RoastCoin where his mouth is.

User Stories

Achyut - Ideas, Investments and Profiles. Collections: ideas, backs

1. Nate - As a user I want to pitch my idea with a title description and category so the community can see it and start debating.
2. Nate - As a user I want to edit or delete my ideas so I can improve them or remove ones I changed my mind about.
3. Marco - As a user I want to browse ideas and sort them by newest or most invested or most roasted or ending soon and filter by category so I can find interesting ones to engage with.

4. Marco - As a user I want to invest RoastCoin in ideas I believe in and see who else backed them so I can support ideas with real stakes.
5. Nate - As a user I want to see a verdict after 7 days showing if my idea survived or got torched so I know what the community thinks.
6. Priya - As a user I want to see profiles showing stats like ideas pitched and ideas torched and RoastCoin balance so I can see how active and credible someone is.

Soni - Roasts and Defenses. Collections: roasts, defenses

1. Priya - As a user I want to write a roast on an idea explaining why it wont work so I can give honest feedback.
2. Priya - As a user I want to edit or delete my roasts so I can fix mistakes or take back something I no longer agree with.
3. Marco - As a user I want to defend an idea by replying to a specific roast with a counter argument so I can help good ideas survive.
4. Marco - As a user I want to edit or delete my defenses so I can strengthen my arguments or remove weak ones.
5. Priya - As a user I want to upvote the best roasts and defenses so the strongest arguments rise to the top and influence the final verdict.

Design Mockups

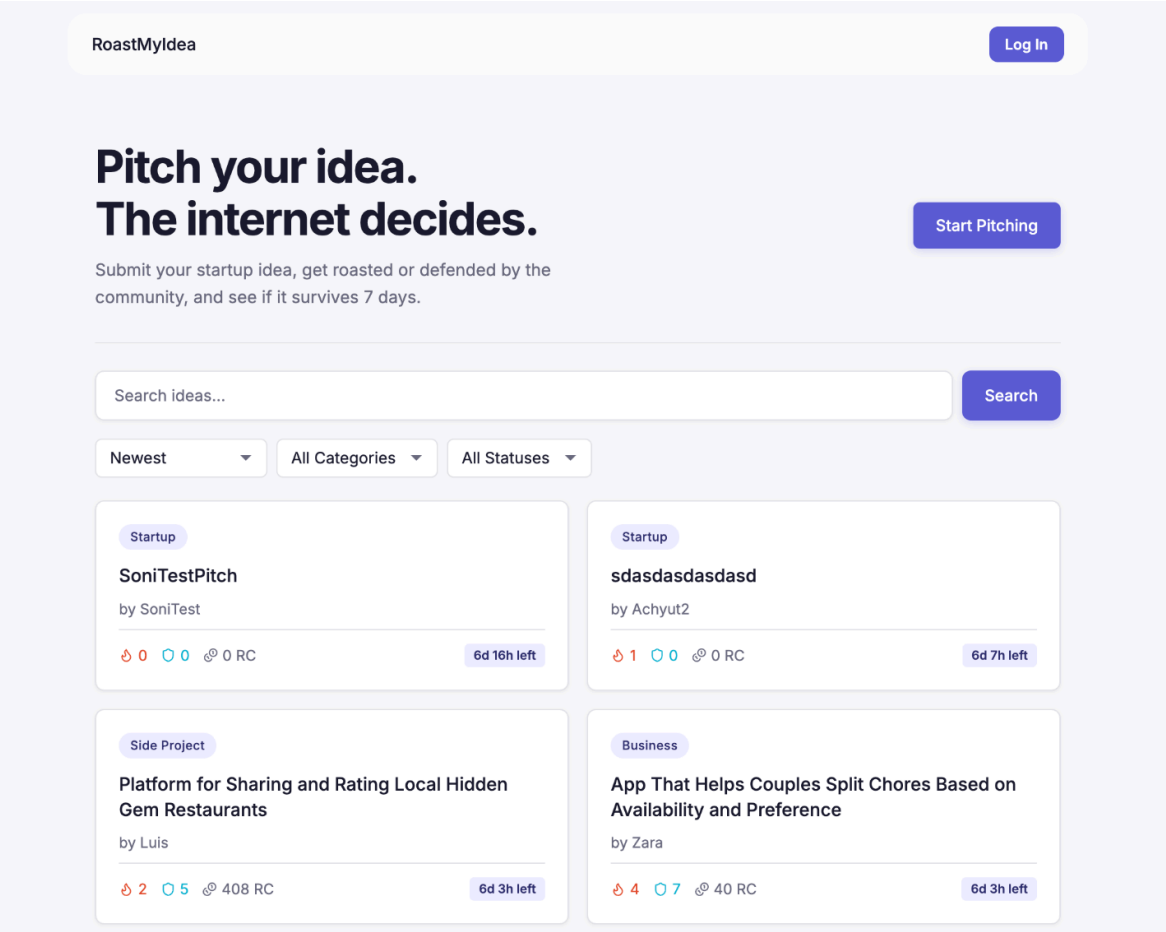
View Wireframes on Figma: [Link to Design]

Preview Link: [Link to preview]

The wireframes cover the following screens, with additional screen details described below:

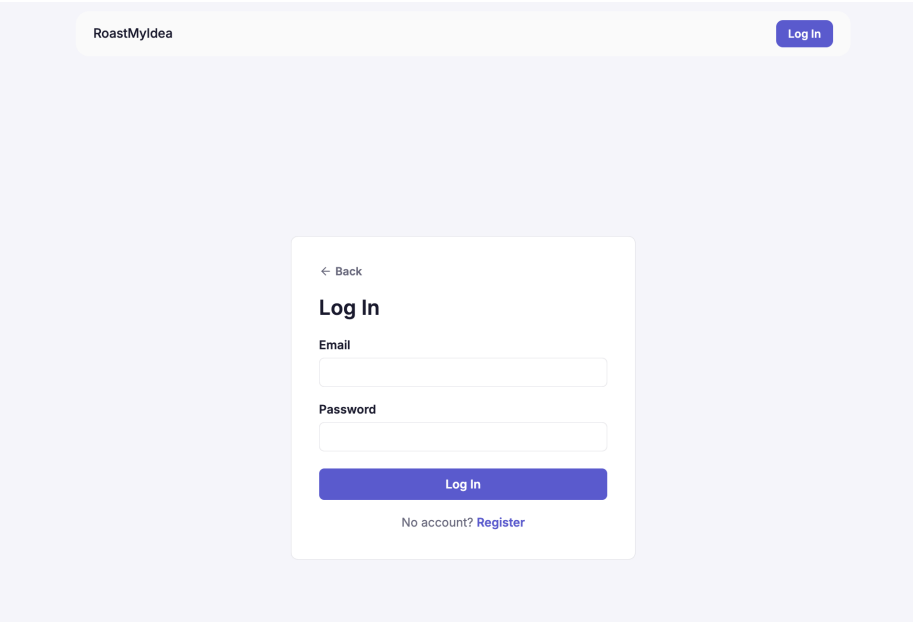
1. Browse Page

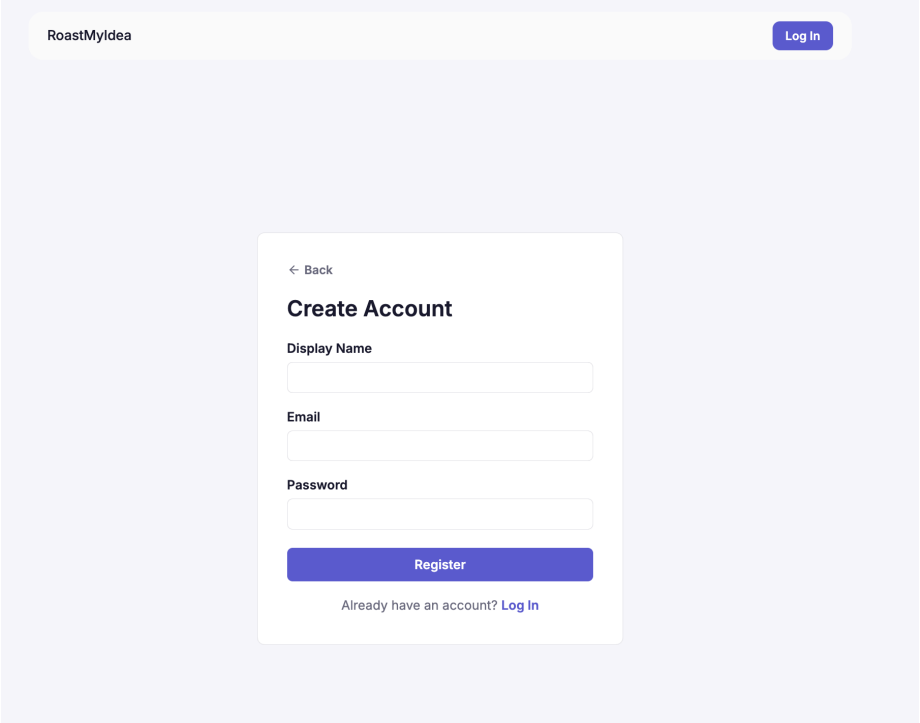
The default view showing the floating pill navbar with the RoastMyIdea logo and navigation links. Below is a hero section with a bold headline reading Pitch your idea. The internet decides. and a Pitch Your Idea button. Below the hero is a search bar, filter dropdowns for sort, category, and status, and a responsive 2 column grid of idea cards. Each card shows a category pill, idea title, author name, and a footer row with roast count, defense count, RC invested, and a verdict or countdown badge.



2. Auth Page

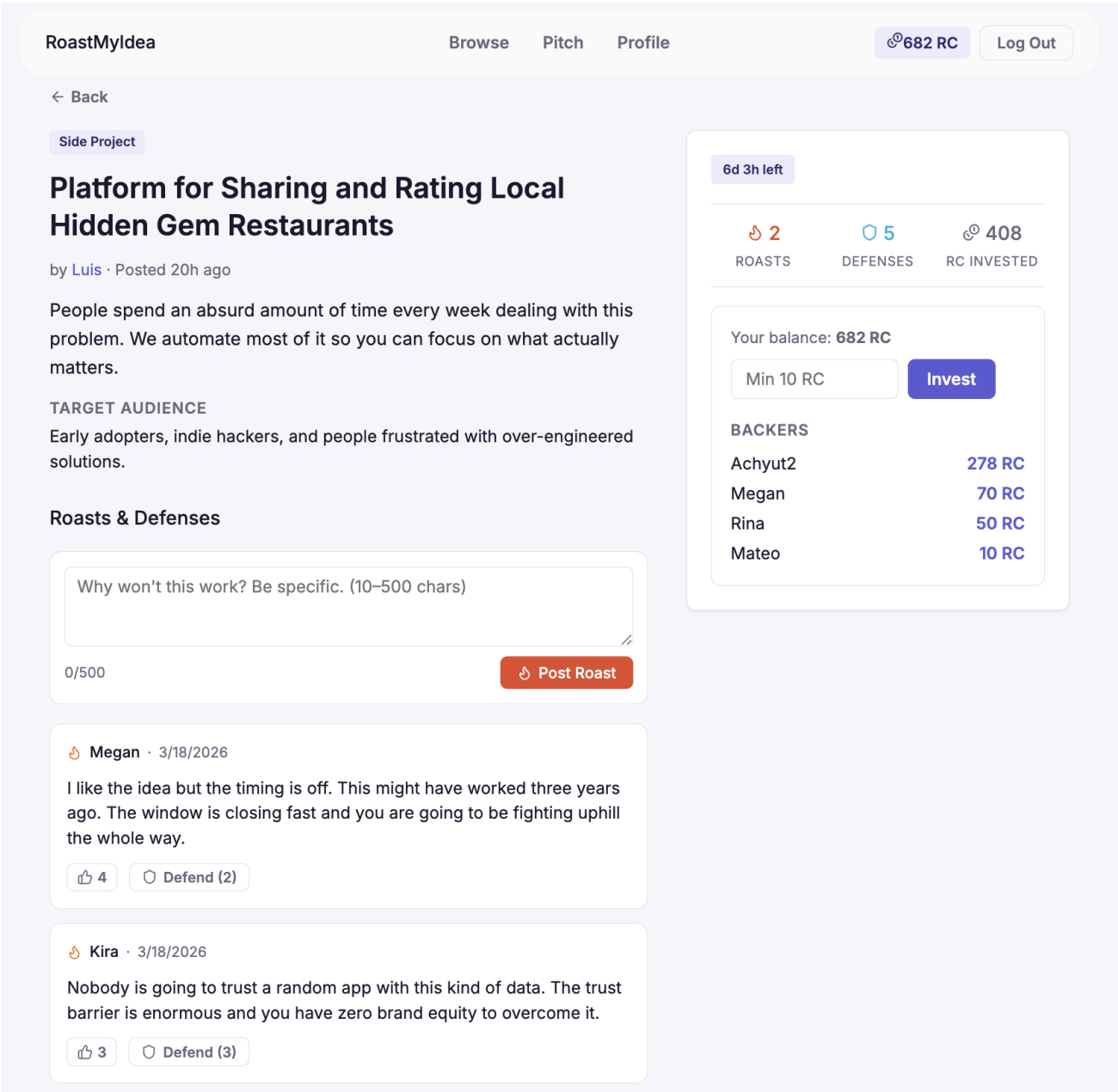
A centered card with the RoastMyIdea logo. Contains a toggle between Log In and Register. The Log In form shows Email and Password fields with a Log In button. The Register form shows Display Name, Email, Password, and Confirm Password fields with a Register button. A back button in the top left returns to browse.





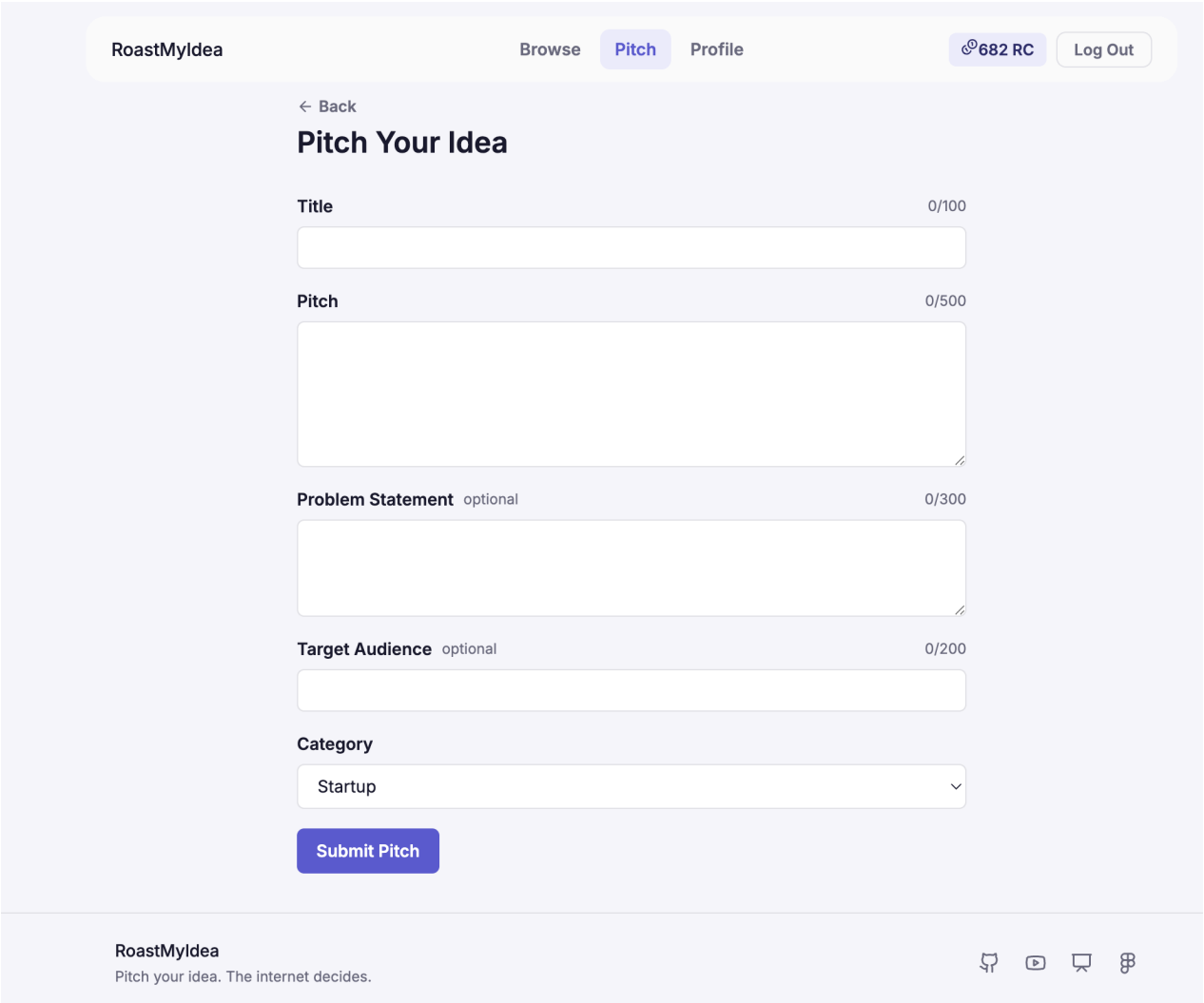
3. Idea Detail Page

A two column layout on desktop. The left column shows the back button, category pill, idea title, author and posted time, pitch content, problem statement, and target audience. Below the content is the roasts and defenses section. The right sidebar is sticky and shows the verdict or countdown badge, a stats grid with Roasts, Defenses, and RC Invested, the invest widget with balance and amount input, and the backers list.



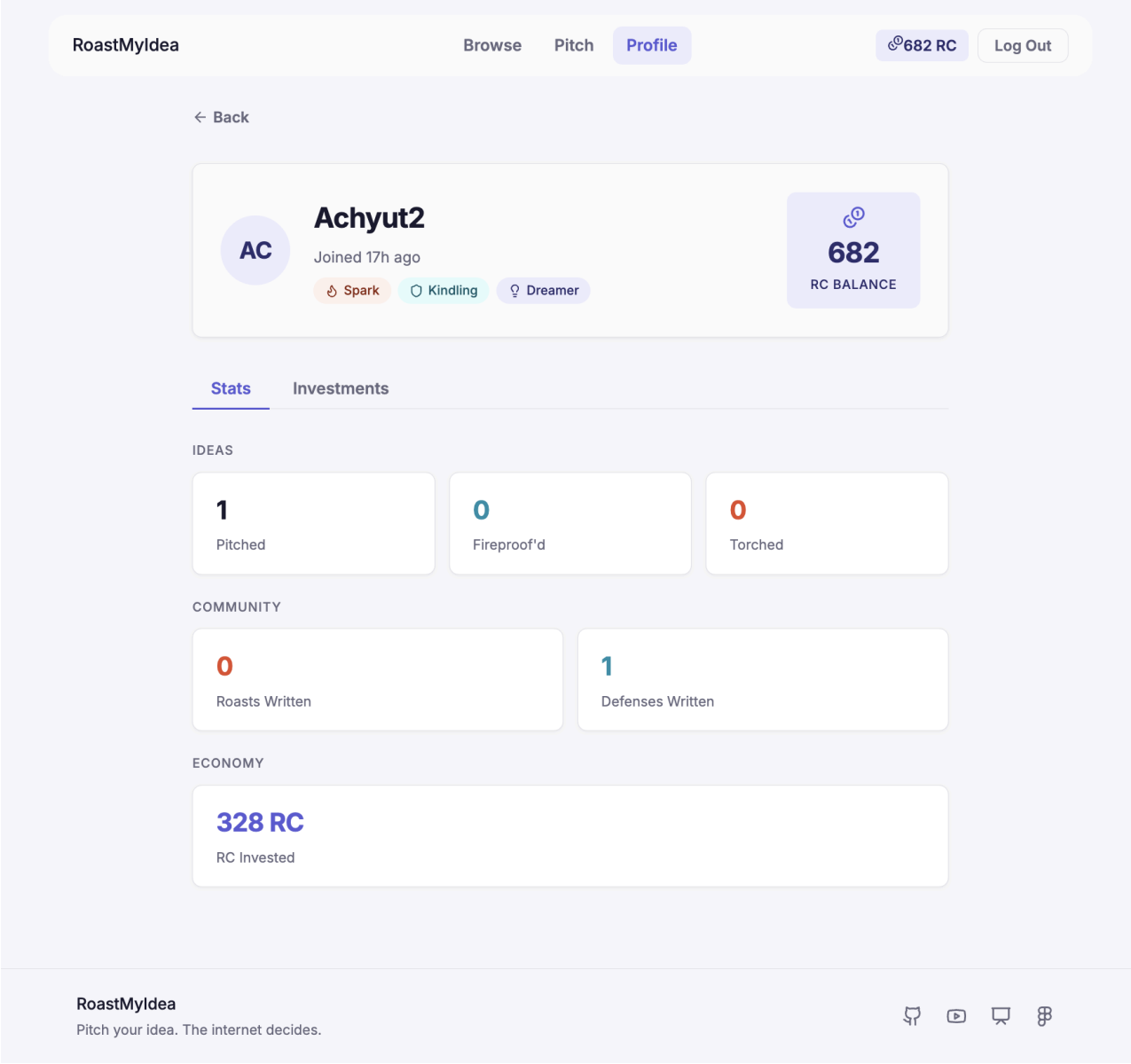
4. Pitch Page

A single column form with the page title Pitch Your Idea and a back button. Fields include Title with a character counter, Pitch textarea, optional Problem Statement textarea, optional Target Audience input, and a Category dropdown. A Submit Pitch button at the bottom.



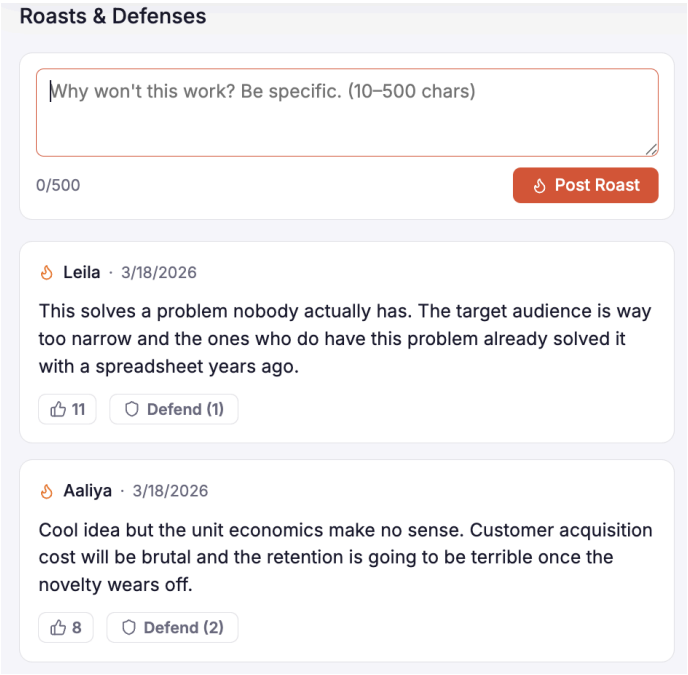
5. Profile Page

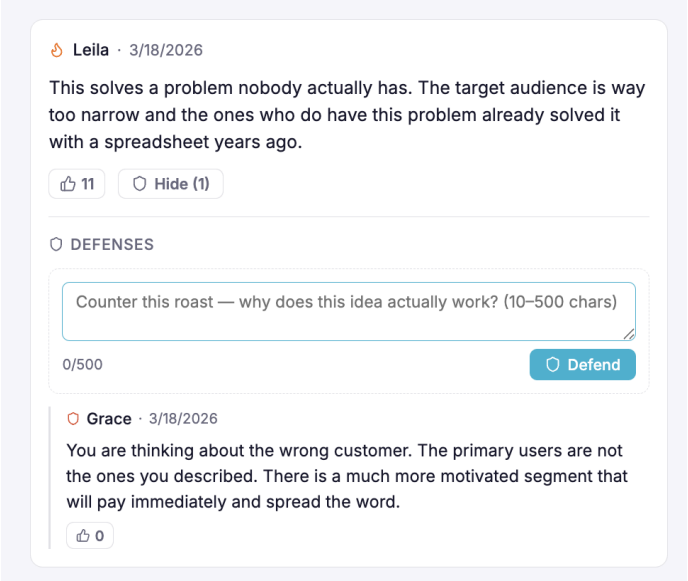
A hero card showing the user avatar with initials, display name, joined date, title badges for roaster defender and pitcher, and a prominent RC Balance block. Below is a tab bar with Stats and Investments. The Stats tab shows three grouped sections: Ideas with pitched fireproof and torched counts, Community with roasts and defenses written, and Economy with RC invested. The Investments tab shows a list of backed ideas with total amount and verdict status.



6. Roast and Defense Section

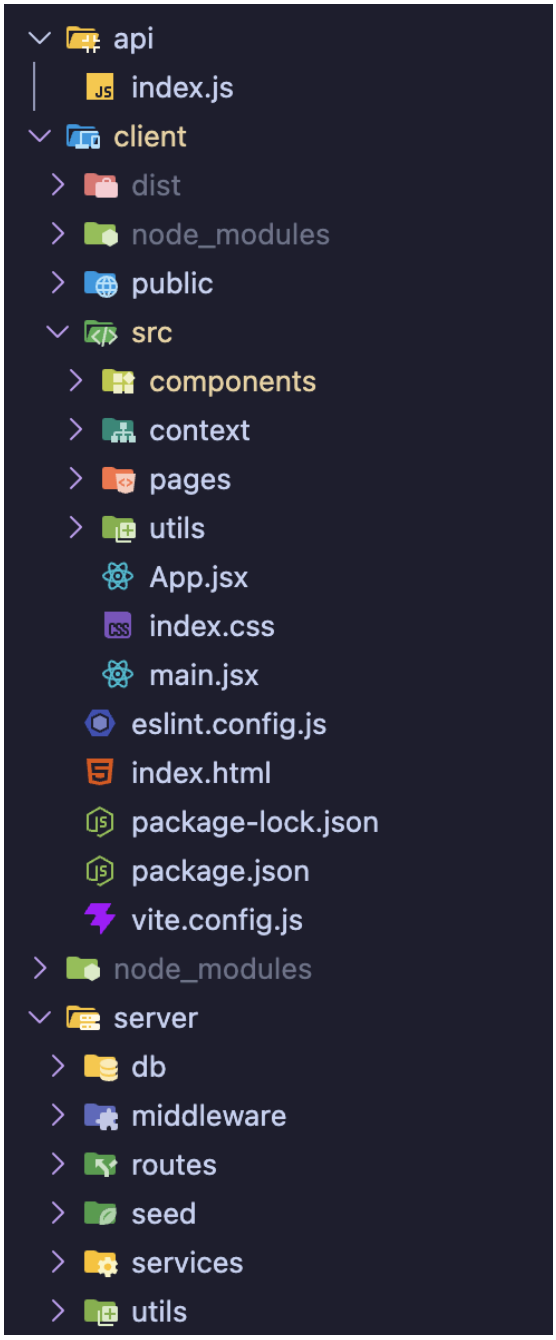
Shown on the idea detail page below the left column content. A section title Roasts and Defenses. A textarea to write a roast with a character counter and a Post Roast button in coral. Each roast card shows the author name, date, content, a thumbs up upvote button, and a Defend button in teal that expands a defense thread below the roast. The defense thread shows a textarea to write a defense and all existing defenses with upvote buttons.





Technical Decisions

Folder Architecture



The backend is organized under server/ with separate folders for db, middleware, routes, services, seed, and utils. The frontend is under client/src with components organized by feature into auth, backs, ideas, profile, roasts, and shared folders. Each component has its own CSS file with the same name. Pages are in a pages/ folder and context providers are in context/. The entry point for the API is api/index.js which wraps Express as a Vercel serverless function.

Tech Stack

The backend uses Node.js with Express in ESM module format for routing and middleware. MongoDB was chosen as the database because the data is document oriented and the schema is flexible. The native MongoDB driver is used instead of Mongoose to keep the code simple and avoid unnecessary abstraction. Authentication uses Passport.js with a JwtStrategy for protecting routes and a LocalStrategy for handling login. Passwords are hashed with bcrypt. The frontend is built with React 18 and Vite using hooks throughout. Lucide React is used for all icons.

Architecture

The app follows a client side rendering pattern. The Express server exposes a REST API and Vite builds the React frontend into static files. All rendering happens in the browser using React components and hooks. The frontend uses a context system where AuthContext manages the JWT token and user state and ToastContext manages notifications. Each feature area has its own folder of components so roasts, backs, ideas, and profile each live in their own directory.

Authentication

Passport.js is used with two strategies. The LocalStrategy handles email and password login by verifying credentials against the database. The JwtStrategy protects all authenticated routes by extracting the Bearer token from the Authorization header and looking up the user. On login the server returns a JWT that the client stores and sends on every authenticated request. The requireAuth middleware exported from the passport module is used directly in routes.

Database Design

Five collections are used: users, ideas, roasts, defenses, and backs. The author display name is denormalized onto ideas and roasts to avoid extra lookups when rendering feeds. Counts like roastCount, defenseCount, and totalRoastCoinInvested are denormalized onto ideas and updated with $inc on every create and delete. This avoids counting documents on every GET request. The backs collection stores individual investment records so the verdict service can calculate payouts correctly.

Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
backs	-	53.25 kB	400	95.00 B	4	184.32 kB
defenses	-	122.88 kB	1.1K	378.00 B	6	516.10 kB
ideas	-	69.63 kB	152	619.00 B	9	376.83 kB
roasts	-	147.46 kB	599	394.00 B	4	303.10 kB
users	-	36.86 kB	32	196.00 B	3	110.59 kB

Search and Pagination

Ideas use keyset pagination with a compound sort on the primary sort field and _id as a tiebreaker. This produces stable pages even when new ideas are added. Full text search uses a MongoDB $text index on the title and pitch fields with title weighted at 10 and pitch at 5 so title matches rank higher. When a search query is present results are sorted by text score instead of the usual sort field.

CSS Architecture

CSS is split into component level files where every component that requires styling has its own CSS file with the same name loaded in the component file. Global variables for colors, spacing, and typography are defined in index.css using CSS custom properties. The Inter font is self hosted from the public/fonts directory using @font-face declarations. No CSS frameworks are used.

Deployment

The app is configured for Vercel serverless deployment. The Express app is wrapped as a serverless function in api/index.js. The React frontend is built by Vite into a static dist folder. The vercel.json config routes /api/* requests to the serverless function and everything else to the static build. The database is hosted on MongoDB Atlas so it is accessible from both local development and production.